

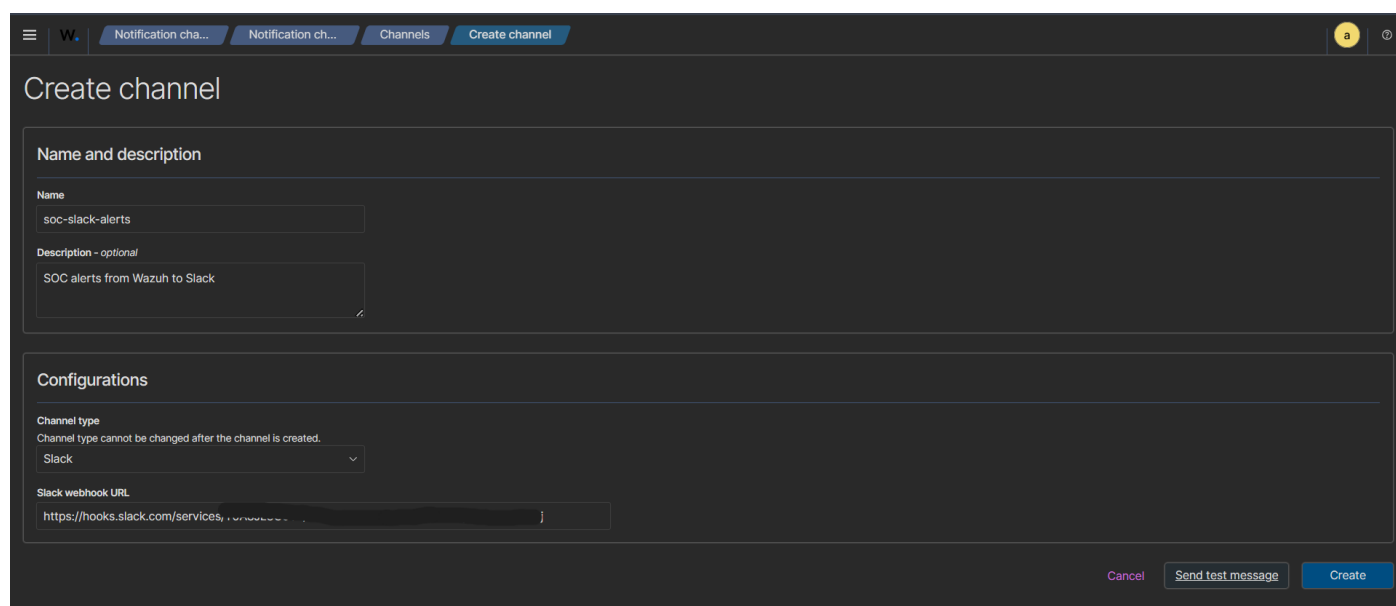
SSH Brute Force Detection and Alerting using Wazuh, Slack, and Kali Linux

➤ Project Objective

The objective of this project is to simulate an SSH brute force attack in a controlled lab environment and configure Wazuh to detect the attack and send real-time alerts to a Slack channel. This project demonstrates end-to-end SOC alerting, from attack simulation to analyst notification.

Step 1: Creating Alert Notification Channel in Wazuh

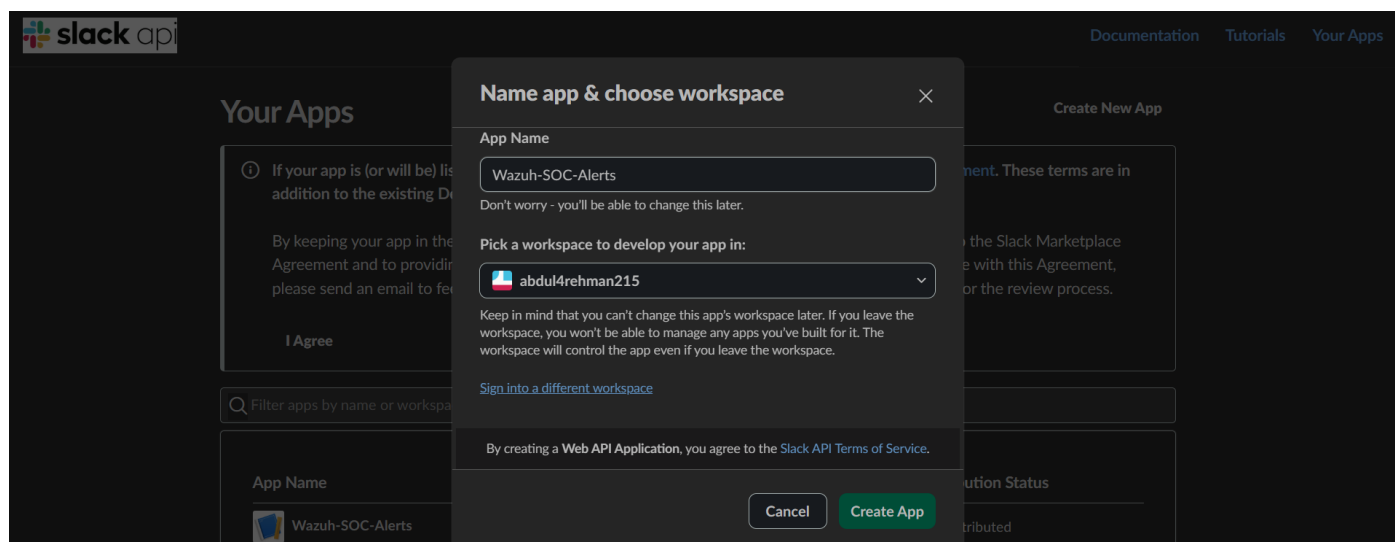
To enable alert delivery, the first step was to create a dedicated notification channel in Wazuh. This channel is responsible for sending security alerts from Wazuh to an external communication platform.



The screenshot shows the 'Create channel' interface in the Wazuh web console. The top navigation bar includes 'Notification cha...', 'Notification ch...', 'Channels', and 'Create channel'. The main form is titled 'Create channel' and is divided into two sections: 'Name and description' and 'Configurations'. In the 'Name and description' section, the 'Name' field is filled with 'soc-slack-alerts' and the 'Description - optional' field is filled with 'SOC alerts from Wazuh to Slack'. In the 'Configurations' section, the 'Channel type' is set to 'Slack' (with a note that it cannot be changed after creation) and the 'Slack webhook URL' is filled with 'https://hooks.slack.com/services/...'. At the bottom right, there are three buttons: 'Cancel', 'Send test message', and 'Create'.

Step 2: Setting Up Slack App for SOC Alerts

A new Slack application was created specifically for SOC alerting purposes. This app is used to receive alerts from Wazuh and post them into a Slack channel in real time.



The screenshot shows the Slack 'Name app & choose workspace' dialog box. The 'App Name' field is filled with 'Wazuh-SOC-Alerts'. Below it, there is a note: 'Don't worry - you'll be able to change this later.' The 'Pick a workspace to develop your app in:' section shows a dropdown menu with 'abdul4rehman215' selected. Below this, there is a warning: 'Keep in mind that you can't change this app's workspace later. If you leave the workspace, you won't be able to manage any apps you've built for it. The workspace will control the app even if you leave the workspace.' There is a link 'Sign into a different workspace'. At the bottom, there is a note: 'By creating a Web API Application, you agree to the Slack API Terms of Service.' and two buttons: 'Cancel' and 'Create App'.

slackapi

DocumentationTutorialsYour Apps

Your Apps

Create New App

1

If your app is (or will be) listed in the Slack Marketplace, please review our [Slack Marketplace Agreement](#). These terms are in addition to the existing Developer Policy, API TOS, and Brand Guidelines.

By keeping your app in the Slack Marketplace or review process, you're confirming your agreement to the Slack Marketplace Agreement and to providing additional information for security review, if requested. If you don't agree with this Agreement, please send an email to feedback@slack.com, and we'll remove your app from the Slack Marketplace or the review process.

I Agree

Filter apps by name or workspace

App Name	Workspace	App ID	App Type	Distribution Status
Wazuh-SOC-Alerts	abdul4rehman215		Modern	Not distributed

3: Enabling Incoming Webhooks in Slack

Incoming Webhooks were enabled in the Slack application to allow Wazuh to send alert messages. A webhook URL was generated, which acts as the endpoint for receiving alerts.

slackapi

DocumentationTutorialsYour Apps

Wazuh-SOC-AL...

Settings

Basic Information

Collaborators

Socket Mode

Install App

Manage Distribution

Features

App Home

Agents & AI Apps NEW

Work Object Previews...

Workflow Steps NEW

Org Level Apps

Incoming Webhooks

Interactivity & Shortcuts

Slash Commands

Steps from Apps LEGACY

OAuth & Permissions

Event Subscriptions

User ID Translation

App Manifest NEW

Incoming Webhooks

Activate Incoming Webhooks

On

Incoming webhooks are a simple way to post messages from external sources into Slack. They make use of normal HTTP requests with a JSON payload, which includes the message and a few other optional details. You can include [message attachments](#) to display richly-formatted messages.

Adding incoming webhooks requires a bot user. If your app doesn't have a bot user, we'll add one for you.

Each time your app is installed, a new Webhook URL will be generated.

If you deactivate incoming webhooks, new Webhook URLs will not be generated when your app is installed to your team. If you'd like to remove access to existing Webhook URLs, you will need to [Revoke All OAuth Tokens](#).

Webhook URLs for Your Workspace

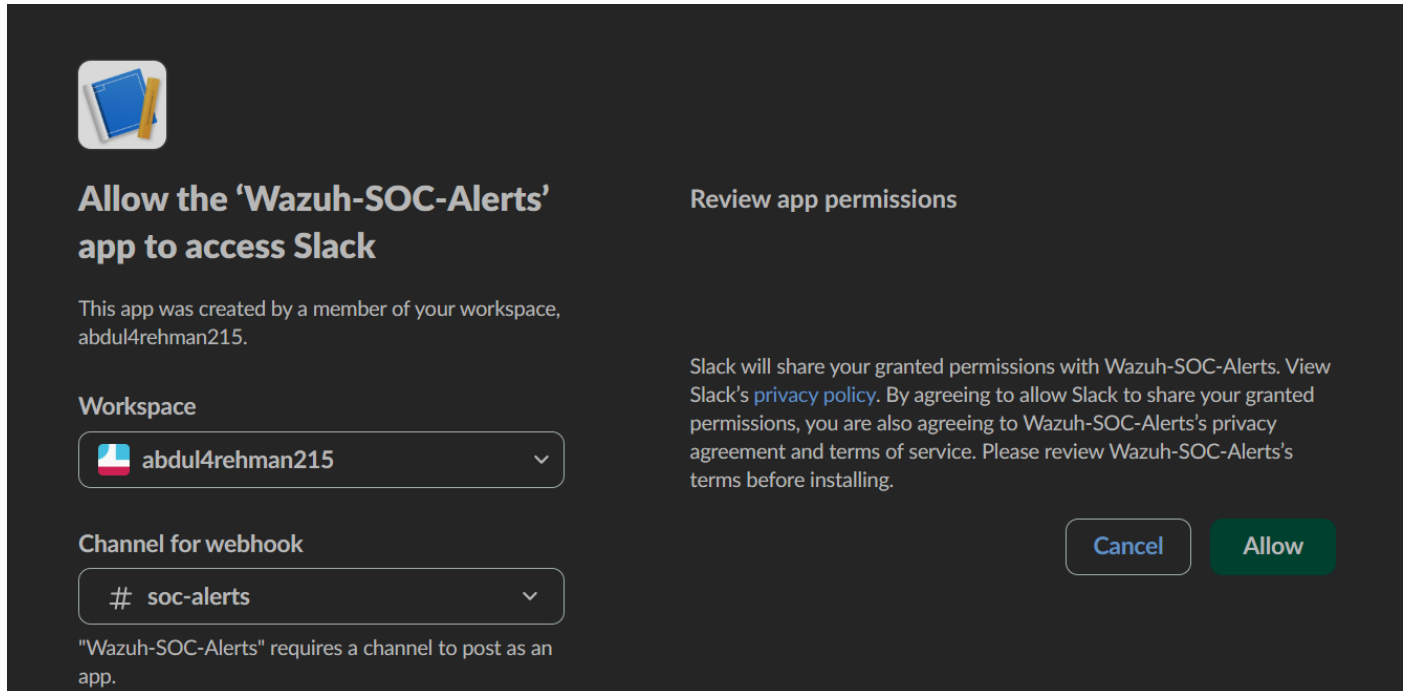
To dispatch messages with your webhook URL, send your [message](#) in JSON as the body of an `application/json` POST request.

Add this webhook to your workspace below to activate this curl example.

Sample curl request to post to a channel:

Step 4: Granting Slack Permissions and Selecting Alert Channel

The Slack app was authorized to access the workspace and post messages to the dedicated SOC alerts channel. This ensures alerts are visible to analysts immediately.



The screenshot shows the Slack permission interface for the 'Wazuh-SOC-Alerts' app. On the left, there's a header 'Allow the 'Wazuh-SOC-Alerts' app to access Slack' with a sub-header 'Review app permissions'. Below this, it states 'This app was created by a member of your workspace, abdul4rehman215.' There are two dropdown menus: 'Workspace' set to 'abdul4rehman215' and 'Channel for webhook' set to '# soc-alerts'. A note at the bottom says '"Wazuh-SOC-Alerts" requires a channel to post as an app.' On the right, there's a paragraph explaining that Slack will share granted permissions with the app and linking to the privacy policy. At the bottom right are 'Cancel' and 'Allow' buttons.

Allow the 'Wazuh-SOC-Alerts' app to access Slack

Review app permissions

This app was created by a member of your workspace, abdul4rehman215.

Workspace

abdul4rehman215

Channel for webhook

soc-alerts

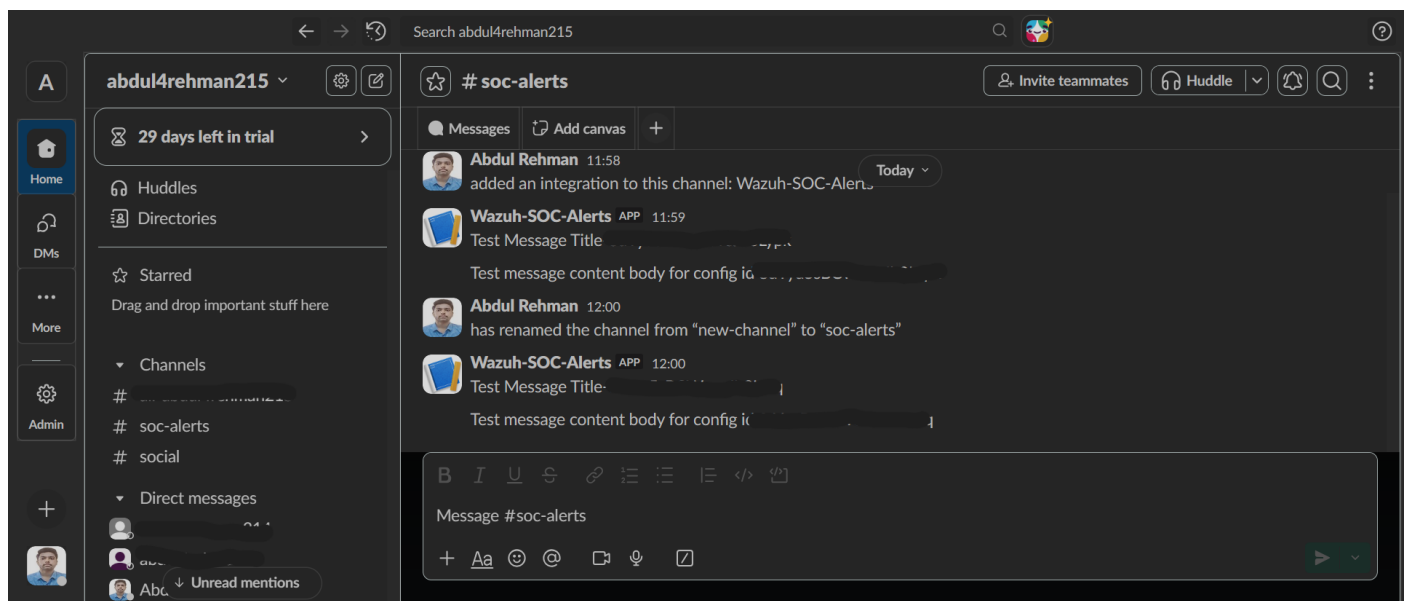
"Wazuh-SOC-Alerts" requires a channel to post as an app.

Slack will share your granted permissions with Wazuh-SOC-Alerts. View Slack's [privacy policy](#). By agreeing to allow Slack to share your granted permissions, you are also agreeing to Wazuh-SOC-Alerts's privacy agreement and terms of service. Please review Wazuh-SOC-Alerts's terms before installing.

Cancel Allow

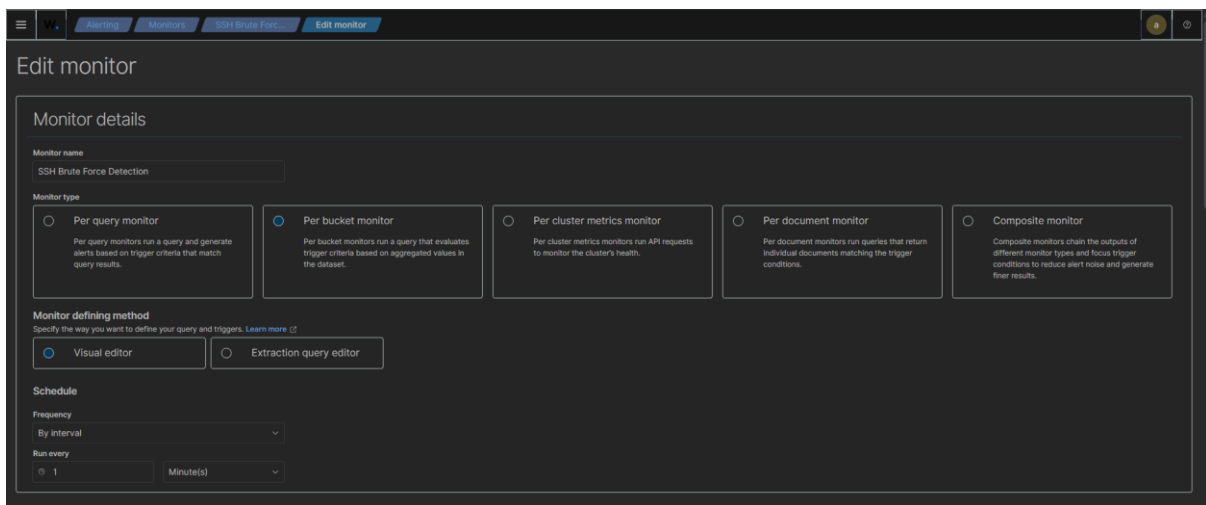
Step 5: Verifying Slack Alert Channel Connectivity

Test messages were successfully received in the Slack channel, confirming that the integration between Wazuh and Slack was working correctly.



Step 6: Creating SSH Brute Force Detection Monitor in Wazuh

A detection monitor was created in Wazuh to identify SSH brute force activity. The monitor was configured to analyze SSH authentication failure events collected from the client machine.



The screenshot shows the 'Edit monitor' interface in Wazuh. The monitor name is 'SSH Brute Force Detection'. The monitor type is 'Per bucket monitor'. The monitor defining method is 'Visual editor'. The schedule is set to 'By interval' with a frequency of '1' minute(s).

Monitor details

Monitor name: SSH Brute Force Detection

Monitor type:

- ☐ Per query monitor: Per query monitors run a query and generate alerts based on trigger criteria that match query results.
- ☒ Per bucket monitor: Per bucket monitors run a query that evaluates trigger criteria based on aggregated values in the dataset.
- ☐ Per cluster metrics monitor: Per cluster metrics monitors run API requests to monitor the cluster's health.
- ☐ Per document monitor: Per document monitors run queries that return individual documents matching the trigger conditions.
- ☐ Composite monitor: Composite monitors chain the outputs of different monitor types and focus trigger conditions to reduce alert noise and generate finer results.

Monitor defining method:

Specify the way you want to define your query and triggers. [Learn more](#)

- ☒ Visual editor
- ☐ Extraction query editor

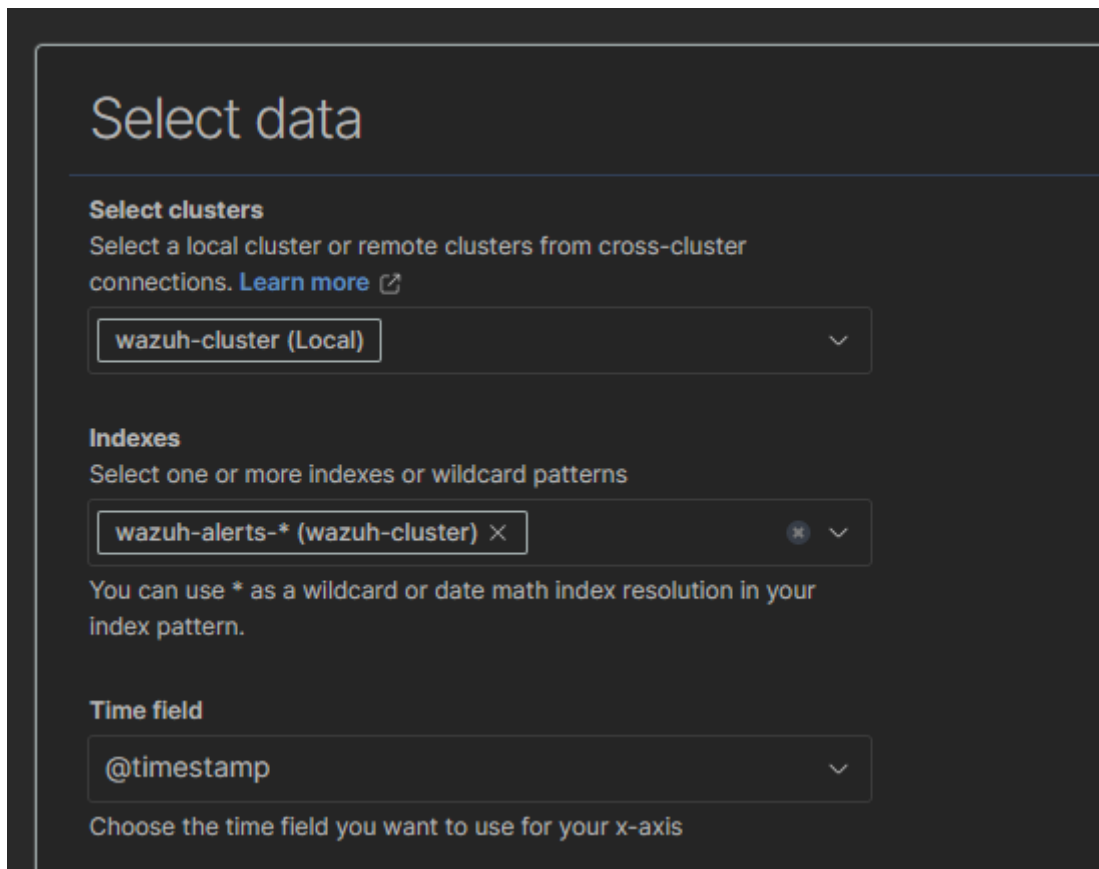
Schedule:

Frequency: By interval

Run every: 1 Minute(s)

Step 7: Selecting Wazuh Alert Data Source

The monitor was configured to read data from Wazuh alert indexes and use the timestamp field for time-based analysis.



The screenshot shows the 'Select data' interface in Wazuh. The 'Select clusters' section shows 'wazuh-cluster (Local)' selected. The 'Indexes' section shows 'wazuh-alerts-* (wazuh-cluster)' selected. The 'Time field' section shows '@timestamp' selected.

Select data

Select clusters
Select a local cluster or remote clusters from cross-cluster connections. [Learn more](#)

wazuh-cluster (Local)

Indexes
Select one or more indexes or wildcard patterns

wazuh-alerts-* (wazuh-cluster)

You can use * as a wildcard or date math index resolution in your index pattern.

Time field

@timestamp

Choose the time field you want to use for your x-axis

Step 8: Defining Detection Logic for SSH Brute Force

The detection logic was configured to:

- Count SSH authentication failure events
- Filter by SSH brute force rule ID
- Consider only higher severity events
- Detect multiple failures within a short time window

Step 9: Grouping Events by Attacker Source IP

Events were grouped by source IP address to correctly identify brute force attempts coming from the same attacker instead of treating each event individually.

The screenshot shows a 'Query' configuration window with the following sections:

- Metrics - optional** ⓘ
 - Box: COUNT OF documents
 - + Add metric
 - You can add up to 5 more metrics.
- Time range for the last** ⓘ
 - Box: 1
 - Dropdown: minute(s) ▼
- Data filter - optional** ⓘ
 - Box: rule.id is 100300 ×
 - Box: rule.level is greater than equal 5 ×
 - + Add filter
 - You can add up to 8 more data filters.
- Group by** ⓘ
 - Box: data.srcip ×
 - + Add group by
 - You can add up to 1 more group by.

Step 10: Configuring Alert Trigger Threshold

An alert trigger was configured to fire when the number of SSH authentication failures exceeded a defined threshold within one minute. This helps reduce false positives while still detecting real attacks.

Triggers (1)

SSH Brute Force Alert

Trigger name

SSH Brute Force Alert

Severity level

2 (High)

Trigger conditions

Specify thresholds for the metrics you have chosen in your query above.

Metric

Count of documents

Threshold

IS ABOVE

Value

5

+ Add condition

You can add up to 9 more trigger conditions.

Keyword filter - optional ?

No filters defined.

+ Add filter

You can add up to 1 keyword filter.

Step 11: Linking Slack Notification Action to Alert

The previously created Slack notification channel was attached to the alert so that whenever the brute force condition is met, an alert is sent automatically to Slack.

Actions (1)

Define actions when trigger conditions are met.

Notification: soc-slack-alerts

Action name

soc-slack-alerts

Names can only contain letters, numbers, and special characters

Channels

[Channel] soc-slack-alerts

Manage channels

Message subject

Alerting Notification action

Message

Embed variables in your message using Mustache templates. [Learn more](#)

Monitor {{ctx.monitor.name}} just entered alert status. Please investigate the issue.
- Trigger: {{ctx.trigger.name}}
- Severity: {{ctx.trigger.severity}}
- Period start: {{ctx.periodStart}} UTC

☐ Preview message

Message preview

Monitor SSH Brute Force Detection just entered alert status. Please investigate the issue.
- Trigger: SSH Brute Force Alert
- Severity: 2
- Period start: 2026-01-14T08:14:41Z UTC

Action configuration

Perform action

☐ Per execution

☒ Per alert

Actionable alerts

De-duplicated

New

Throttling

☐ Enable action throttling

Step 12: Simulating SSH Brute Force Attack from Kali Linux

An SSH brute force attack was simulated from the Kali Linux attacker machine by repeatedly attempting SSH logins using invalid usernames against the client machine.

```
(kali@kali)-[~]
$ for i in {1..10}; do ssh fakeuser@10.0.1.10; done

fakeuser@10.0.1.10: Permission denied (publickey).
fakeuser@10.0.1.10: Permission denied (publickey).
fakeuser@10.0.1.10: Permission denied (publickey).
fakeuser@10.0.1.10: Permission denied (publickey).
fakeuser@10.0.1.10: Permission denied (publickey).
fakeuser@10.0.1.10: Permission denied (publickey).
fakeuser@10.0.1.10: Permission denied (publickey).
fakeuser@10.0.1.10: Permission denied (publickey).
fakeuser@10.0.1.10: Permission denied (publickey).
fakeuser@10.0.1.10: Permission denied (publickey).
```

Step 13: Detection and Alert Generation in Wazuh

Wazuh successfully detected the repeated SSH authentication failures, correlated them as a brute force attack, and triggered the alert based on the configured logic.

Alerts

Alerts by triggers

Search

All severity levels

All alerts

Alerts	Active	Acknowledged	Errors	Trigger name	Trigger start time	Trigger last updated	Severity	Monitor name
1 alert	1	0	0	SSH Brute Force Alert	01/14/26 1:26 pm	01/14/26 1:26 pm	2	SSH Brute Force Detection

SSH Brute Force Detection

Overview

Monitor type
Per bucket monitor

Monitor definition type
Visual Graph

Index
wazuh-alerts-*

Total active alerts
0

Schedule
Every 1 minutes

Last updated
01/14/26 1:25 pm IST

Monitor ID
B50Kus8BCWewJh3BSu

Monitor version number
3

Associations with composite monitors

Triggers (1)

Name	Number of actions	Severity
SSH Brute Force Alert	1	2

History

SSH Brute Force Alert

01/12/2026 12:00 AM

01/14/2026 01:28 PM

Triggered

No alerts

Alerts

Search

All severity levels

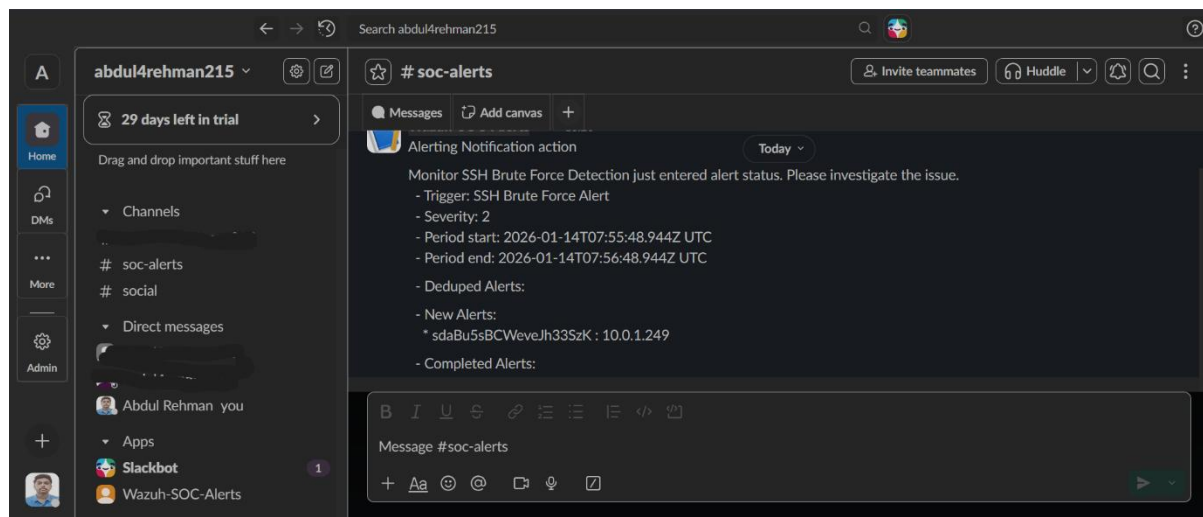
All alerts

Alert start time	Alert last updated	State	Trigger name	Severity	Data srcip
01/14/26 1:26 pm	01/14/26 1:27 pm	Completed	SSH Brute Force Alert	2	-

Rows per page: 20

Step 14: Real-Time Alert Received in Slack

The SSH brute force alert was successfully delivered to the Slack SOC alerts channel in real time, including details such as alert name, severity, time window, and attacker IP.



❖ Outcome

This project successfully demonstrates a complete SOC detection and alerting workflow. An SSH brute force attack was simulated, detected by Wazuh, and escalated in real time via Slack. The setup reflects real-world SOC operations, including alert engineering, attack detection, and incident notification.

✓ Skills Demonstrated

- Wazuh SIEM configuration
- SSH brute force attack detection
- SOC alert engineering
- Slack integration for security alerts
- Linux log analysis
- Blue team defensive security practices